The solution must distinguish three situations:

- ✓ A timeout: you do not know whether the service accepted the request.
- ✓ A rate limit: the service is asking you to slow down, often indicated by HTTP 429.
- ✓ An invalid email: the request should be rejected before calling the service.

The most important concept here is idempotency: retrying the same logical request should not create a second signup.

Step 1: Understand the reliable approach

The preferred design is to use an idempotency key that the newsletter service explicitly supports. Reuse the same key for every retry of the same signup operation.

If the provider does not support idempotency, a local database record alone cannot guarantee exactly-once delivery when a timeout occurs after the provider has accepted the request. You need a provider-supported deduplication mechanism or a reconciliation strategy.

Step 2: Example JavaScript implementation

This example assumes the provider supports idempotency keys and exposes a subscription endpoint. The endpoint and request format are illustrative and must be adapted to the real provider

```
async function subscribeToNewsletter(email, {
  apiUrl,
  apiKey,
  idempotencyKey,
  saveAttempt,
  fetchImpl = fetch,
}) {
  const normalizedEmail = String(email).trim().toLowerCase();

  // 1. Validate before making a network request.
```

```
const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

if (!emailPattern.test(normalizedEmail)) {
  return {
    ok: false,
    status: "invalid_email",
  };
}

if (!idempotencyKey) {
  throw new Error("An idempotency key is required.");
}

// 2. Persist a pending attempt without storing secrets.
await saveAttempt({
  idempotencyKey,
  email: normalizedEmail,
  status: "pending",
});

// 3. Retry transient failures only.
const maxAttempts = 3;

for (let attempt = 1; attempt <= maxAttempts; attempt++) {
  let response;

  try {
    response = await fetchImpl(apiUrl, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "Authorization": `Bearer ${apiKey}`,
        "Idempotency-Key": idempotencyKey,
      },
      body: JSON.stringify({
        email: normalizedEmail,
      }),
      signal: AbortSignal.timeout(5000),
```

```
  });
} catch (error) {
  // A timeout is ambiguous: the provider may have accepted it.
  await saveAttempt({
    idempotencyKey,
    email: normalizedEmail,
    status: "unknown",
    lastError: error.name === "TimeoutError"
      ? "timeout"
      : "network_error",
  });

  if (attempt === maxAttempts) {
    return {
      ok: false,
      status: "unknown",
      retryable: true,
    };
  }

  // The same key must be reused on every retry.
  continue;
}

if (response.ok) {
  await saveAttempt({
    idempotencyKey,
    email: normalizedEmail,
    status: "confirmed",
  });

  return {
    ok: true,
    status: "confirmed",
  };
}

if (response.status === 429) {
```

```
  const retryAfter = Number(
    response.headers.get("Retry-After") || 1
  );

  await saveAttempt({
    idempotencyKey,
    email: normalizedEmail,
    status: "rate_limited",
  });

  // Do not hammer the service with immediate retries.
  return {
    ok: false,
    status: "rate_limited",
    retryAfterSeconds: Number.isFinite(retryAfter)
      ? Math.max(1, retryAfter)
      : 1,
  };
}

if (response.status >= 500) {
  await saveAttempt({
    idempotencyKey,
    email: normalizedEmail,
    status: "provider_error",
  });

  if (attempt < maxAttempts) {
    continue;
  }

  return {
    ok: false,
    status: "provider_error",
    retryable: true,
  };
}
```

```
    // Other 4xx errors should not be retried blindly.
    await saveAttempt({
      idempotencyKey,
      email: normalizedEmail,
      status: "rejected",
      httpStatus: response.status,
    });

    return {
      ok: false,
      status: "rejected",
      httpStatus: response.status,
    };
  }

  return {
    ok: false,
    status: "unknown",
  };
}
```

Step 3: Understand the implementation

- ✓ The email is normalized and checked before the network request.
- ✓ A pending attempt is persisted so that the application can track what happened.
- ✓ The same idempotency key is sent on every retry.
- ✓ A timeout is recorded as unknown, because the remote service might already have accepted the request.
- ✓ A rate-limit response is handled separately, respecting the provider's retry instructions.
- ✓ Server errors may be retried within a limit.
- ✓ Permanent client errors are not repeatedly retried.

This is a compact illustration, not a complete production retry system. A production implementation would need bounded exponential backoff with jitter, durable retry scheduling, and a defined recovery process for requests whose final outcome

remains unknown. It should also check the provider's precise idempotency and rate-limit semantics.

Step 4: What should be stored or logged?

Store:

- ✓ A stable operation or idempotency key.
- ✓ The normalized email, with appropriate access controls.
- ✓ The attempt status and timestamps.
- ✓ The provider's subscription identifier, if returned.
- ✓ A safe error category and HTTP status.

Avoid logging:

- ✓ API keys or authorization headers.
- ✓ Full request headers containing secrets.
- ✓ Unnecessary copies of personal data.
- ✓ Complete request or response bodies when they could contain sensitive information.

Email addresses are personal data, so access, retention, and log visibility should be limited.

Step 5: How to test the difficult timeout case

Use a mock service that records the idempotency key, accepts the first request, but deliberately delays the response until the client times out.
Then retry with the same key.
Expected result: the mock records only one logical subscription, and the second request returns the original result or a provider-defined equivalent rather than creating another subscription.
Also test invalid email input, HTTP 429, repeated server errors, and the situation where the provider accepted the request but the local status update failed.